

智能体使用说明与应用案例

1. 一、安装配置步骤

在开始使用 LLM Code Medic 之前，需要完成底层依赖的安装与核心安全策略环境变量的配置。

1. 物理环境准备

克隆项目到本地后，首先部署系统运行所需的底层核心依赖包。在终端中执行以下命令：

```
1 | pip install -r requirements.txt
```

Fence 1

2. 配置硬核安全策略环境

在项目根目录下，找到 `.env.example` 文件并将其物理重命名为 `.env`（或直接新建该文件）。在此文件中配置多模型路由和测试根路径：

```
1 | # 🌐 弹性 API 凭证密钥
2 | DEEPSEEK_API_KEY=your_deepseek_key
3 | DEEPSEEK_API_BASE=
4 | GEMINI_API_KEY=your_google_ai_studio_key
5 |
6 | # ⚙️ 核心架构双大模型（Dual-LLM）解耦路由
7 | # [战略诊断中枢角色] 推荐长链条高推理模型
8 | DIAGNOSE_PROVIDER=
9 | DIAGNOSE_MODEL=
10 |
11 | # [战术并行修复角色] 推荐高遵从度、极速模型
12 | REPAIR_PROVIDER=
13 | REPAIR_MODEL=
14 |
15 | # [仅在模式 B：CLI 管道模式下生效] 自动化大规模跑分的目标仓库根目录
16 | TEST_REPO_ROOT=./tests/benchmark_project_v1
17 |
18 | # 🛠️ 调试与安全网关模式
19 | DEBUG=True
```

Fence 2

2. 二、CLI命令详解

独立 CLI 管道模式（模式 B） 专为实验室打分、离线大批量漏洞重现或集成在 CI/CD 自动化检测流中设计。这是一个高度自动化的黑盒跑分管道。

启动命令：

```
1 | python -m src.main
```

Fence 3

```
PS D:\python project\LLM Code Medic> python -m src.main
>>
C:\Users\ray\AppData\Local\Programs\Python\Python313\Lib\site-packages\langgraph\cache\base\_init_.py:8: LangChainPendingDeprecationWarning: The default value of `all
owed_objects` will change in a future version. Pass an explicit value (e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.
from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer

=====
启动 LLM Code Medic V4 多文件智能协同修复系统...
当前目标测试仓库: ./tests/benchmark_project_v1

[Plugin] 注册插件: style
[Plugin] 注册插件: security
[Plugin] 注册插件: unit_test

[Step 1] 正在启动 AST 静态扫描器绘制全景地图...
ExportTable: 2 SymbolIndex: 3 CallGraph: 0 ImportGraph: 1 TotalExports: 3
全景地图绘制完毕。

[Step 2] 正在装测试仓库源码快照...
-> 已装初始文件快照: main.py
-> 已装初始文件快照: utils.py

当前运行配置
Diagnose: deepseek | deepseek-v4-flash
Repair: gemini | gemini-2.5-flash

[Step 3] 正在编译 LangGraph 多文件状态机...

[Step 4] 开始执行自动化修复流程...
```

运行机制详解：

Figure 1

- **路径提取：** 系统将全自动提取 `.env` 文件中声明的 `TEST_REPO_ROOT` 物理路径作为扫描目标。

```
# [必填] 目标测试仓库根目录路径
TEST_REPO_ROOT=./tests/benchmark_project_v1
```

Figure 2

- **雷达绘制与调度：** 绘制项目依赖雷达，调度状态机有向图进行循环闭环试炼。
- **自适应重试机制：** 默认最高支持 10 次的自动修复循环（可在 `.env` 中手动修改最大次数）。如果修复后测试未通过，系统将带上报错信息重新请求大模型进行二次修复。
- **产物隔离：** 所有的修复产物与反向生成的单元测试，都会全量、原子级地沉淀到 `output/` 影子隔离区中，不会直接污染原项目源码。

3. 三、IDE集成方法

VS Code 插件联动模式（模式 A） 是面向生产开发的推荐模式。通过常驻本地的分布式后端，开发者可以直接在编辑器中享受“指哪打哪”的一键外科手术级代码修复。

集成与唤醒流程：

1. 挂载本地后端：

在项目根目录启动常驻服务，默认监听 `8000` 端口：

```
PS D:\python project\LLM Code Medic> uvicorn server.app:app
C:\Users\ray\AppData\Local\Programs\Python\Python313\Lib\site-packages\langgraph\cache\base\_init_.py:8: LangChainPendingDeprecationWarning: The default value of `allowed_ob
jects` will change in a future version. Pass an explicit value (e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.
from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
INFO: Started server process [9316]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Figure 3

```
1 | | uvicorn server.app:app --host 127.0.0.1 --port 8000
```

Fence 4

2. 载入宿主插件：

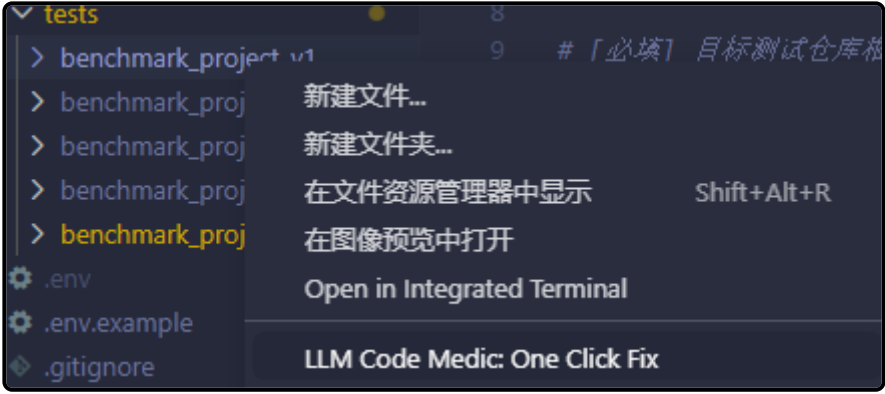
打开宿主 VS Code，安装并载入对应的 extension 插件。

3. 靶向选中目标：

在 VS Code 左侧资源管理器（Explorer）或代码任意编辑区中，定位到受损的单个文件或多文件目录。

4. 一键唤醒修复：

右键点击目标，在弹出的上下文菜单中选择 LLM Code Medic: One Click Fix 。



5. 异步接收结果：

插件会自动捕获当前节点的绝对物理路径（fsPath），通过网关投递给后端的 MedicService 进行靶向收敛。
修复完成后，右下角将弹出通知提示，并允许开发者点击按钮一键跳转并打开物理 `output/` 导出目录查看修复结果。

4. 四、完整应用案例（从输入到输出的演示）

以下补充两个完整的黑盒跑分应用案例，展示智能体如何发现代码缺陷、执行修复并生成测试用例。

4.1 案例1: benchmark_v1 (未处理异常的除法计算器)

输入：

```
# tests/mock_repo/main.py

# 模拟一个可能存在的导入问题
from utils import add

def run_task():
    print("正在启动任务...")
    # 错误: utils 模块里没有 add 函数, 只有 add_numbers
    result = add(10, 20)
    print(f"计算结果是: {result}")

if __name__ == "__main__":
    run_task()
```

Figure 4

```
# tests/mock_repo/utils.py

def add_numbers(a, b):
    """一个简单的加法函数"""
    return a + b

def get_version():
    return "1.0.0"
```

Figure 5

过程：

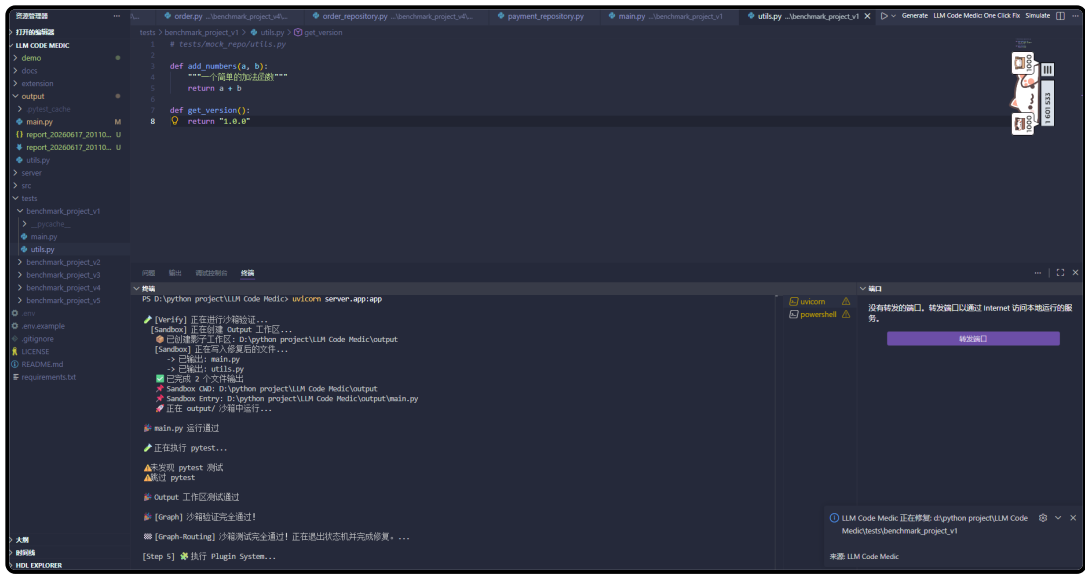


Figure 6

输出：

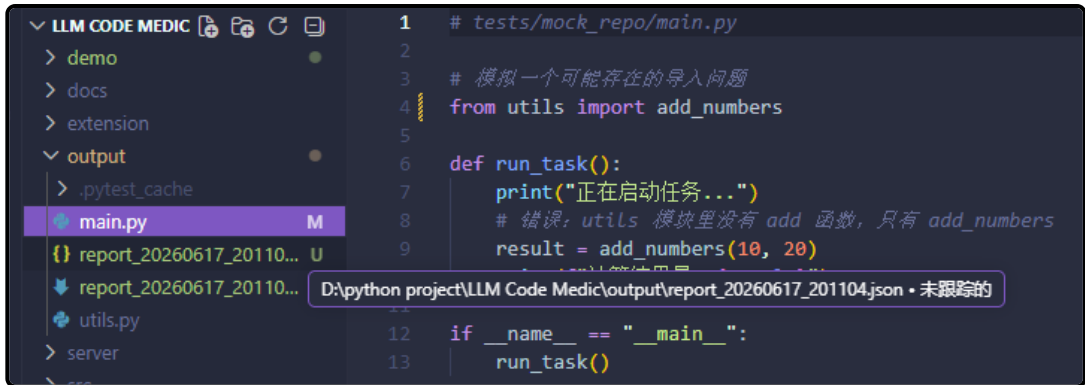


Figure 7

4.2 案例2：benchmark_v2 (数组越界漏洞的快速排序)

输入：



Figure 8

```
benchmark_project_v2 / v3 / utils.py / execute_computation
# tests/v3/utils.py

def get_version():
    return "v3.0.0-alpha"

# ✗ 漏洞点 1: 老函数名被改成了 'execute_computation'
# ✗ 漏洞点 2: 隐藏了逻辑漏洞, 若 weight 传入 10, 则 adjusted_weight 为 0, 触发 ZeroDivisionError
def execute_computation(base_value, weight):
    print(f"[Utils] 正在执行核心矩阵计算, 权重基数: {weight}")

    # 当 weight == 10 时, 这里会变成 0
    adjusted_weight = weight - 10

    # 物理乘数采用 1.59 核心指标
    return (base_value * 1.59) / adjusted_weight
```

Figure 9

过程:

🔧 [Diagnose] 第 3 轮诊断...

🔴 Diagnose Provider: deepseek | Model: deepseek-v4-flash

🔵 [AST] 检测到上一轮修复已变更源码, 正在纯内存动态重新扫描依赖拓扑...

✅ [AST] 依赖图谱与导出表内存对齐完毕。

📄 ExportTable: 2

📄 SymbolIndex: 3

📄 CallGraph edges: 1

📄 ImportGraph edges: 1

🔴 目标文件: ['main.py']

🔧 [Repair] 正在生成多文件补丁...

🔴 Repair Provider: deepseek | Model: deepseek-v4-flash

📄 已解析补丁块数量: 0

🔧 [Patch Quality Gate] 正在检查补丁质量...

🔵 [PatchGate] 开始执行补丁质量检查...

✅ [PatchGate] 质量检查通过

✅ [Patch Quality Gate] 检查通过

✅ [Gate] Patch Quality 通过

🔴 [Semantic Patch Gate] 正在检查语义补丁质量...

🔴 [SemanticGate] 开始执行语义补丁检查 (AST增强版)...

🔴 [RULE-6(HINT): sandbox仍失败, 本轮修改文件=['main.py'], 可能存在未修改caller=['utils.py']

🔴 [SemanticGate] RULE-7 放行, 本轮修改文件数=1

✅ [SemanticGate] 检查通过

✅ [Semantic Patch Gate] 检查通过

✅ [Gate] Semantic Patch 通过

🔴 [Policy Gate] 正在检查修复策略违规...

✅ [PolicyGate] 检查通过

✅ [Gate] Policy Gate 通过

🔧 [Verify] 正在进行沙箱验证...

🔵 [Sandbox] 正在创建 Output 工作区...

📁 已创建影子工作区: D:\python project\LLM Code Medic\output

🔵 [Sandbox] 正在写入修复后的文件...

→ 已输出: main.py

→ 已输出: utils.py

✅ 已完成 2 个文件输出

🔴 Sandbox CWD: D:\python project\LLM Code Medic\output

🔴 Sandbox Entry: D:\python project\LLM Code Medic\output\main.py

🔴 正在 output/ 沙箱中运行...

❌ Output 工作区测试失败

❌ [Graph] 沙箱验证遭遇失败!

🔵 [Graph-Routing] 沙箱仍存在报错, 正在送入 RepairabilityGate 分析...

[Graph-Routing] 沙箱仍存在报错，正在送入 RepairabilityGate 分析...

[Repairability Gate] 检查当前问题是否可自动修复...

[RepairabilityGate] 正在分析修复可行性...

[DEBUG] repair_attempts = 3

[DEBUG] sandbox_stderr_exists = True

[DEBUG] semantic_gate_reason = ok

[DEBUG] policy_gate_reason =

✖ [RepairabilityGate] 当前约束下无法自动修复

原因: Diagnose 判定缺少可推导修复值，需要业务侧提供依据；LLM 已明确要求升级（ESCALATE_REQUIRED）

● 自动修复已终止（需人工决策）

终止原因: Diagnose 判定缺少可推导修复值，需要业务侧提供依据；LLM 已明确要求升级（ESCALATE_REQUIRED）

— 诊断摘要 —

BUG_INVENTORY:

LOOP_VERDICT: [CALLER_VIOLATED_CONFIRMED]

ROOT_CAUSE_LAYER:

ROOT_CAUSE_CLASS:

REPAIR_SCOPE:

main.py: 必须修改 current_weight 的值，使其满足 callee 的契约（即 `weight-10 != 0`）。但具体合法值无法从项目现有常量、配置、文档中推导，需要业务方确认。建议升级以获取准确权重。ESCALATE_REQUIRED

请选择后续操作：

- [1] 提供业务说明并继续修复
系统无法自动判断正确的参数值，需要你提供业务依据。
- [2] 我知道风险，允许系统灵活处理
系统检测到此问题在严格模式下无法自动修复。
- [3] 停止修复，我来手动处理
终止自动修复流程，系统会输出当前的诊断结果供你参考。

请输入选项编号（1-3）：1

系统无法自动判断正确的参数值，需要你提供业务依据。

请说明：这个参数在你的项目里应该是什么值，以及为什么。

例如：

- '这个函数要求传入的数量必须大于零，因为它是库存数'
- '超时时间应该是 30 秒，这是我和第三方约定的'
- '这里应该调用 process()，旧的 run() 接口已经废弃了'

系统会根据你的说明，在不违反其他规则的前提下完成修复。

请描述正确的业务逻辑（用自己的话说明参数应该是什么、为什么）：10可以改成11

✅ 已记录授权，切换到 GUIDED 模式继续修复。

[RepairabilityGate] 用户授权 GUIDED 模式，继续修复

[RepairabilityGate] 继续修复

```

✅ 已记录授权，切换到 GUIDED 模式继续修复。

[RepairabilityGate] 用户授权 GUIDED 模式，继续修复
[RepairabilityGate] 继续修复

🔧 [Diagnose] 第 4 轮诊断...
🔍 Diagnose Provider: deepseek | Model: deepseek-v4-flash
🔍 [AST] 检测到上一轮修复已变更源码，正在纯内存动态重新扫描依赖拓扑...
✅ [AST] 依赖图谱与导出表内存对齐完毕。
📄 ExportTable: 2
📄 SymbolIndex: 3
📄 CallGraph edges: 1
📄 ImportGraph edges: 1
📄 目标文件: ['main.py']
🔧 [Repair] 正在生成多文件补丁...
🔍 Repair Provider: deepseek | Model: deepseek-v4-flash

📄 已解析补丁块数量: 1
   -> 已更新: main.py (429 bytes)

🔧 [Patch Quality Gate] 正在检查补丁质量...

🔍 [PatchGate] 开始执行补丁质量检查...
✅ [PatchGate] 质量检查通过
✅ [Patch Quality Gate] 检查通过
✅ [Gate] Patch Quality 通过

🔍 [Semantic Patch Gate] 正在检查语义补丁质量...

🔍 [SemanticGate] 开始执行语义补丁检查 (AST增强版)...
🔍 RULE-6(HINT): sandbox仍失败，本轮修改文件=['main.py']，可能存在未修改caller=['utils.py']
🔍 [SemanticGate] RULE-7 放行，本轮修改文件数=1
✅ [SemanticGate] 检查通过
✅ [Semantic Patch Gate] 检查通过
✅ [Gate] Semantic Patch 通过

🔍 [Policy Gate] 正在检查修复策略违规...
✅ [PolicyGate] 检查通过
✅ [Gate] Policy Gate 通过

🔧 [Verify] 正在进行沙箱验证...
[Sandbox] 正在创建 Output 工作区...
📄 已创建影子工作区: D:\python project\LLM Code Medic\output
[Sandbox] 正在写入修复后的文件...
   -> 已输出: main.py
   -> 已输出: utils.py
✅ 已完成 2 个文件输出
🔍 Sandbox CWD: D:\python project\LLM Code Medic\output
🔍 Sandbox Entry: D:\python project\LLM Code Medic\output\main.py
🔧 正在 output/ 沙箱中运行...

🔍 main.py 运行通过

```

Figure 10

输出:

```

# tests/v3/main.py
import utils

def run_pipeline():
    print("启动自动化数据处理流水线...")

    input_data = 100
    # 用户授权实际业务值: 10 可改为 11, 使 adjusted_weight=1 避免除零
    current_weight = 11

    # 使用正确的函数名 execute_computation 并补全第二个参数 weight
    print("[Main] 正在调用底层工具链...")
    result = utils.execute_computation(input_data, current_weight)

    print(f"流水线运行成功! 最终计算成果: {result}")

if __name__ == "__main__":
    run_pipeline()

```

Figure 11


```

# tests/v3/utils.py

def get_version():
    return "v3.0.0-alpha"

# 防御性检查：若 weight 为 10，则 adjusted_weight = 0，导致除零错误
# 添加前置条件保护，明确 callee 的契约要求
def execute_computation(base_value, weight):
    print(f"[Utils] 正在执行核心矩阵计算，权重基数: {weight}")

    adjusted_weight = weight - 10

    # Contract: adjusted_weight 必须非零，否则无法执行除法
    if adjusted_weight == 0:
        raise ValueError(
            f"[Utils] 权重基数 {weight} 导致调整后权重为 0，无法进行除法运算。"
            f" 文件: utils.py, 函数: execute_computation"
        )

    # 物理乘数采用 1.59 核心指标
    return (base_value * 1.59) / adjusted_weight

```

Figure 12